

转转集团 前端开发1-3面

一面

1. 你先自我介绍一下吧。
2. 我看你简历上写了 XXX 项目，介绍一下这个项目，你主要负责什么？
3. 说一下你对闭包的理解，它有什么应用场景？
4. JavaScript 的原型和原型链是怎么回事？
5. `Promise` 有几种状态？`Promise.all` 和 `Promise.allSettled` 有什么区别？
6. 讲一下 Event Loop，宏任务和微任务的执行顺序。
7. Vue3 的响应式原理是什么？`ref` 和 `reactive` 在使用上有什么区别？
8. `v-if` 和 `v-show` 的区别和使用场景。
9. Vue 里面 `keep-alive` 是做什么用的？它的生命周期是怎样的？
10. Vue 的 diff 算法有了解吗？key 的作用是什么？
11. Webpack 和 Vite 用起来有什么区别？Vite 为什么快？
12. Webpack 的 Loader 和 Plugin 有什么不同？你自己写过吗？
13. 从输入 URL 到页面完全加载，中间发生了什么？
14. 了解浏览器的强缓存和协商缓存吗？

转转集团 前端开发 二面

1. 挑一个你简历里你觉得最复杂的项目，说一下它的背景、你负责的模块、遇到的难点。
2. 你刚说这个难点，你是怎么定位和解决的？
3. 前端性能优化你一般从哪些方面入手？
4. `LCP`、`FCP`、`CLS` 这些指标分别指什么？你怎么去监控和优化？
5. 如果一个列表页数据量特别大，一次性加载很慢，你会怎么优化？
6. 什么是虚拟滚动？它的实现原理是什么？

7. 看 JD 了吧？我们 C 端业务对 SEO 有要求。你之前做过 SEO 优化吗？具体做了哪些？
8. SSR 和 CSR 的区别是什么？
9. 你们团队的通用组件是怎么沉淀的？如何保证组件的质量？
10. 看你用过 TypeScript, `interface` 和 `type` 有什么区别？
11. 移动端 H5 开发和 PC 开发有什么主要差异？怎么做适配？
12. H5 页面白屏问题，你一般怎么排查？
13. 前端工程化方面，除了打包，你还做过哪些事情？比如 CI/CD、代码规范。

参考答案：

一面参考答案

1. 说一下你对闭包的理解，它有什么应用场景？

闭包（Closure）是指一个函数能够“记住”并访问它在定义时所在的词法作用域，即使这个函数在它定义的作用域之外被执行。

通俗地说，就是一个函数（内部函数）“包裹”住了它外部函数（外部函数）的变量，并能持续访问这些变量。

它的形成有两个条件：

1. 一个函数内部定义了另一个函数。
2. 内部函数引用了外部函数的变量。
3. 内部函数在外部被调用（比如被 `return` 出去）。

应用场景：

1. **数据私有化（模拟私有变量）**：这是最常见的用途。我们可以用闭包创建一个模块，只暴露必要的 API（返回的内部函数），而隐藏内部的状态（外部函数的变量）。
 - 例如：创建一个计数器，`count` 变量在外部无法直接修改，只能通过返回的 `increment`、`decrement` 方法来操作。
2. **防抖（Debounce）和节流（Throttle）**：这类高阶函数都利用了闭包来保存定时器 `timer` 和时间戳 `startTime`。
3. **函数柯里化（Currying）**：利用闭包来保存预先传入的参数，返回一个新函数来接收剩余参数。

4. 循环中的事件监听：在 `for` 循环（使用 `var` 的年代）中为DOM元素绑定事件时，会用到闭包（通常是IIFE，立即执行函数）来“锁住”每一次循环的索引值 `i`。
-

5. JavaScript 的原型和原型链是怎么回事？

6. 原型 (Prototype):

- 在 JavaScript 中，对象（`Object`）是核心。函数也是一种对象。
- prototype**：每个函数（Function）都有一个 `prototype` 属性，它指向一个对象。这个对象被称为“原型对象”。这个原型对象的作用是，让所有通过这个函数作为构造函数（`new` 出来的）创建的实例，都能共享它上面的属性和方法。
- __proto__**：每个对象（实例）都有一个 `__proto__` 属性（这是一个非标准但被广泛实现的属性，标准用法是 `Object.getPrototypeOf()`），它指向创建这个实例的构造函数的 `prototype` 对象。

7. 原型链 (Prototype Chain):

- 原型链是 JavaScript 实现继承的机制。
 - 当我们试图访问一个对象的某个属性时（比如 `obj.name`），JavaScript 引擎会首先在 `obj` 对象自身上查找这个属性。
 - 如果找不到，它就会沿着 `obj.__proto__` 指向的原型对象去查找。
 - 如果这个原型对象上还是找不到，它就会再沿着这个原型对象的 `__proto__` 属性继续向上查找。
 - 这个查找过程会一直持续下去，直到找到该属性，或者查找到原型链的顶端——`Object.prototype`。`Object.prototype` 的 `__proto__` 是 `null`。
 - 这条由 `__proto__` 串联起来的对象链条，就叫做原型链。
-

8. Promise 有几种状态？Promise.all 和 Promise.allSettled 有什么区别？

Promise 的三种状态：

- Pending（进行中）**：初始状态，既不是成功，也不是失败。
- Fulfilled（已成功）**：操作成功完成。
- Rejected（已失败）**：操作失败。

状态一旦从 `Pending` 改变为 `Fulfilled` 或 `Rejected`，就凝固了，不会再改变。

`Promise.all` 和 `Promise.allSettled` 的区别：

- `Promise.all`**：
 - 它接收一个 Promise 数组，返回一个新的 Promise。

- “一荣俱荣，一损俱损”。
 - **成功**：只有当数组中 *所有* Promise 都变为 `Fulfilled` 时，`Promise.all` 返回的 Promise 才会 `Fulfilled`，并且 `Fulfilled` 的值是一个按输入顺序排列的结果数组。
 - **失败**：只要数组中 *有任何一个* Promise 变为 `Rejected`，`Promise.all` 返回的 Promise 就会 *立即* `Rejected`，并且 `Rejected` 的原因是第一个失败的 Promise 的原因。
 - **`Promise.allSettled`**：
 - 它也接收一个 Promise 数组，返回一个新的 Promise。
 - “**永远成功**”（除非入参有问题）。它会等待数组中所有的 Promise 都有一个结果（无论是 `Fulfilled` 还是 `Rejected`）。
 - 它返回的 Promise 一旦 `Fulfilled`，其值是一个对象数组，每个对象代表了对应 Promise 的最终状态，格式为：
 - 成功：`{ status: 'fulfilled', value: ... }`
 - 失败：`{ status: 'rejected', reason: ... }`
 - **使用场景**：当你关心 *所有* 异步操作的最终结果，而不想因为其中一个失败就中断所有时，`allSettled` 非常有用。比如，你同时调用多个不相关的 API，希望在页面上展示它们各自的成功或失败状态。
-

4. 讲一下 Event Loop，宏任务和微任务的执行顺序。

JavaScript 是单线程的，但它通过 Event Loop（事件循环）机制来实现异步和非阻塞。

- **执行栈（Call Stack）**：用来执行同步代码。当一个函数被调用，它会被推入栈中；当函数执行完毕，它会从栈中弹出。
- **任务队列（Task Queue）**：异步操作（如 `setTimeout`、DOM 事件、Promise）在完成后，会将其回调函数放入任务队列中。
- **Event Loop**：是一个持续运行的进程，它的工作就是不断地检查执行栈是否为空。如果执行栈为空，它就会去任务队列里取出一个任务，放入执行栈中执行。

宏任务（Macrotask）和微任务（Microtask）：

任务队列被分为两种：

1. 宏任务队列（Macrotask Queue）：

- 包括：`script`（整个脚本）、`setTimeout`、`setInterval`、I/O 操作、UI 渲染。

2. 微任务队列（Microtask Queue）：

- 包括：`Promise.then/catch/finally`、`process.nextTick` (Node.js)、`queueMicrotask`。

执行顺序：

Event Loop 的每一轮“循环”（tick）的执行顺序是固定的：

1. **执行同步代码：** 首先，执行栈会执行完所有的同步代码（比如 `script` 标签内的顶层代码，这本身是一个宏任务）。
2. **清空微任务队列：** 在当前宏任务执行完毕后（比如 `script` 跑完了），Event Loop 会立即检查微任务队列。
3. 它会执行所有在队列中的微任务，直到微任务队列被清空。（重点：是“所有”，如果在执行微任务期间又产生了新的微任务，它们也会被加到队列末尾并在本轮循环中执行）。
4. **执行一个宏任务：** 当微任务队列为空后，Event Loop 会检查宏任务队列，取出一个宏任务（如果队列中有的话），并将其推入执行栈中执行。
5. **重复：** 该宏任务执行完毕后，再次回到第 2 步（清空微任务队列）。

总结： 一轮 Event Loop = 执行 1 个宏任务 -> 执行所有微任务 -> （可能进行UI渲染） -> 开始下一轮循环（取下 1 个宏任务）。

6. Vue3 的响应式原理是什么？ref 和 reactive 在使用上有什么区别？

Vue3 响应式原理：

Vue3 的响应式系统是基于 ES6 的 `Proxy` 实现的。

1. **Proxy 代理：** 当我们使用 `reactive` 或 `ref` 创建一个响应式对象时，Vue 3 会返回一个该对象的 `Proxy` 代理实例。
2. **get 依赖收集 (Track)：** 当组件的 `render` 函数执行并访问响应式对象的属性时（例如 `state.count`），会触发 `Proxy` 的 `get` 陷阱。在这个 `get` 陷阱中，Vue 的响应式系统会“追踪”这个动作，将当前正在执行的 `effect`（通常是组件的渲染函数）作为依赖，收集到该属性的依赖列表中。
3. **set 触发更新 (Trigger)：** 当我们修改响应式对象的属性时（例如 `state.count++`），会触发 `Proxy` 的 `set` 陷阱。在这个 `set` 陷阱中，Vue 会找到所有依赖这个属性的 `effect`（在第 2 步中收集的），并触发它们重新执行（比如，重新渲染组件）。

相比 Vue 2 的 `Object.defineProperty`，`Proxy` 的优势在于它是对整个对象进行代理，可以原生支持新增属性、删除属性和数组的索引访问及 `length` 修改，而不需要像 Vue 2 那样做复杂的“hack”或遍历。

ref 和 reactive 的区别：

- **reactive：**
 - **参数：** 只能接收对象或数组（引用类型）。
 - **原理：** 内部使用 `Proxy` 来创建深度响应式对象。
 - **访问：** 在 `<script setup>` 中，需要通过 `state.property` 访问。

- **问题：** 如果对 `reactive` 对象使用解构赋值，会使其失去响应性。例如 `let { count } = reactive({ count: 1 })`，`count` 会变成一个普通的数字，修改它不会触发更新。
- **ref：**
 - **参数：** 可以接收任何类型的值，包括基本类型（如 `String`、`Number`）和引用类型。
 - **原理：** `ref` 实际上是创建了一个包含 `.value` 属性的对象。响应性是建立在这个 `.value` 属性上的。
 - **访问：** 在 `<script setup>` 中，必须通过 `.value` 来访问和修改（例如 `count.value++`）。
 - **模板中：** 在 `<template>` 中使用时，`ref` 会被自动“解包”，所以你只需要写 `{{ count }}`，而不需要 `{{ count.value }}`。
 - **解决解构问题：** `ref` 解决了 `reactive` 的解构问题。
 - **内部：** 如果 `ref` 接收的是一个对象，它内部实际上会调用 `reactive` 来使这个对象深度响应式。

使用场景总结：

- 当你需要一个响应式的**基本类型**值（字符串、数字、布尔值）时，**必须使用 `ref`**。
- 当你需要一个响应式的**对象或数组**时，**`reactive` 是首选**。
- 如果你想在解构后保持响应性，或者你可能需要重新赋整个对象的值，**使用 `ref`**（例如 `myRef.value = newObject`）。

4. v-if 和 v-show 的区别和使用场景。

- **v-if（条件渲染）：**
 - **原理：** 它是“真正”的条件渲染。如果条件为 `false`，`v-if` 指令的元素及其子组件会被**销毁**并从 DOM 中**完全移除**。如果条件从 `false` 变为 `true`，元素会被**重新创建**、挂载，并经历完整的生命周期。
 - **开销：** 切换时有较高的创建和销毁开销。
 - **使用场景：**
 - i. 条件不经常改变的场景。
 - ii. 当 `v-else` 或 `v-else-if` 需要配合使用时。
 - iii. 为了节省初始渲染开销（如果初始条件为 `false`，它根本不会渲染）。
- **v-show（条件显示）：**
 - **原理：** 无论条件是 `true` 还是 `false`，元素**始终会被渲染**到 DOM 中。
 - `v-show` 只是简单地切换元素的 CSS `display` 属性（`display: none`）。

- **开销：** 有较高的初始渲染开销（因为它总会被渲染），但切换开销非常低（只是修改 CSS）。
- **使用场景：**
 - i. 需要**频繁切换**显示/隐藏状态的场景（例如，Tab 切换）。

总结： `v-if` 是“惰性”的，有更高的切换开销；`v-show` 是“急切”的，有更高的初始渲染开销，但切换开销低。

5. Vue 里面 keep-alive 是做什么用的？它的生命周期是怎样的？

keep-alive 的作用：

`keep-alive` 是 Vue 的一个内置组件，它用于**缓存**动态切换的组件实例。

当一个组件在 `keep-alive` 标签内部被切换时，它的状态（包括数据、DOM）会被保存在内存中，而不是被销毁。当下次再切换回这个组件时，它会从内存中恢复，而不是重新创建。

使用场景：

- 最典型的**就是带缓存的 Tab 标签页**。用户在列表页滚动到了第 100 条，点击进入详情页，再返回列表页时，希望列表页保持在第 100 条的滚动位置，而不是刷新回顶部。
- 保留表单的输入内容。用户填写了一半表单，切换到其他页面，再切回来时，不希望内容丢失。

keep-alive 的生命周期：

被 `keep-alive` 缓存的组件，会多出两个特有的生命周期钩子：

1. `activated`（激活时）：

- 在组件**第一次被创建**时会触发（在 `mounted` 之后）。
- 在组件**从缓存中被激活**（即重新显示）时会触发。
- （注意：此时不会触发 `created` 和 `mounted`）。

2. `deactivated`（失活时）：

- 在组件被切换走并**进入缓存**时触发。
- （注意：此时不会触发 `unmounted`）。

因此，一个被缓存的组件的生命周期可能是：

`created` -> `mounted` -> `activated` -> （切换走） -> `deactivated` -> （切换回来） -> `activated` -> （再切换走） -> `deactivated` ...

3. Vue 的 diff 算法有了解吗？key 的作用是什么？

Diff 算法：

Diff 算法是 Vue 中虚拟 DOM（VNode）更新的核心。当组件状态改变时，Vue 会生成一个新的 VNode 树，然后将新 VNode 树与旧 VNode 树进行比较（即“Diff”），找出两者之间的最小差异，

最后只将这些差异应用（Patch）到真实的 DOM 上，以保证最高效的 DOM 更新。

Vue 的 Diff 算法主要策略是：**同层比较**。它不会跨层级移动 DOM 节点。

在比较同层级的子节点列表时（例如 `v-for` 循环的列表）：

- **Vue 2（双端 Diff）**：它同时比较新旧两个列表的头部和尾部（头头、尾尾、头尾、尾头）。
- **Vue 3（最长递增子序列 - LIS）**：它采用了更高效的策略。
 - a. 先处理相同的“前缀”和“后缀”节点。
 - b. 对中间剩余的乱序节点，它会使用“最长递增子序列”算法来计算出一个索引序列，这个序列代表了最少需要移动的节点。
 - c. 最后，它遍历新列表，如果节点在 LIS 序列中，说明它在正确的位置，不动；如果不在，就将其移动到正确的位置，同时处理新增和删除的节点。

key 的作用：

`key` 是 Diff 算法的核心。

1. **身份标识**：`key` 的主要作用是为 `v-for` 循环中的每个 VNode 提供一个**唯一的、稳定的身份标识**。
2. **高效更新（“就地复用”）**：
 - **没有 `key`**：如果没有 `key`，Vue 会使用“就地复用”策略。它会按顺序比较新旧列表，如果节点类型相同，就复用旧的 DOM 并更新其内容。这在列表只是顺序改变（如“反转”）或增删时，会导致性能低下和状态错乱（比如 `input` 框的输入内容会错位）。
 - **有 `key`**：Vue 会使用 `key` 来跟踪每个节点。它会根据 `key` 快速地识别出哪些节点是**新增**的、**删除**的、还是**移动**了位置。
 - 例如，如果只是交换了两个 `key` 不同的元素的位置，Vue 知道它们只是互换了，会执行 DOM 的**移动**操作，而不是销毁和重建这两个元素，这极大地提高了性能。

总结：`key` 必须是唯一的、稳定的（不能用 `index` 或 `Math.random()`），它帮助 Diff 算法最大限度地复用 DOM 元素，是保证 `v-for` 列表高效、正确更新的关键。

3. Webpack 和 Vite 用起来有什么区别？Vite 为什么快？

核心区别：

最大的区别在于**开发服务器（Dev Server）**的实现机制：

1. Webpack（Bundler-based）：

- “先打包，再启动”。
- 在启动 `webpack-dev-server` 时，Webpack 会从入口文件开始，抓取整个应用的依赖图，然后将所有模块打包（Bundle）到一个或多个文件中（通常是在内存中）。
- 只有打包完成后，服务器才准备就绪。

- **热更新（HMR）**：当你修改一个文件时，Webpack 需要重新打包 *受影响的模块*，即使使用了 HMR，在大型项目中这个过程也会越来越慢。

2. Vite（Native ESM-based）：

- “先启动，再按需编译”。
- Vite 利用了浏览器原生的 **ESM（ES Modules）** 支持。
- **启动时**：Vite 几乎是 *瞬时启动*。它不需要打包，服务启动后，它直接将你的源码（如 `index.html`）提供给浏览器。
- **运行时**：浏览器会自己去解析 `import` 语句。当浏览器请求一个模块（如 `import App from './App.vue'`）时，Vite Dev Server 会拦截这个请求，*按需*对这个文件进行转换（比如把 `.vue` 编译成 JS），然后返回给浏览器。

Vite 为什么快（主要指开发阶段）：

1. **极速的冷启动**：不需要打包，服务秒开。
2. **按需编译**：只有当浏览器请求某个模块时，Vite 才去编译它。不像 Webpack 那样需要提前编译所有模块。
3. **高效的热更新（HMR）**：当一个文件被修改时，Vite 只需要让浏览器重新请求这一个 *被修改的模块* 即可。它利用 ESM 的特性，更新是 *模块级别的*，与项目大小无关，所以 HMR 始终非常快。Webpack 的 HMR 粒度是 *打包链级别* 的。
4. **利用 esbuild**：Vite 在开发阶段使用 `esbuild`（一个 Go 编写的工具）来进行预构建和代码转换，`esbuild` 的速度比 Webpack 使用的 `babel` 和 `terser` 快 10-100 倍。

注意：在 ****生产环境（Production Build）**** 中，Vite 仍然会进行打包。它默认使用 `Rollup`（`esbuild` 用于压缩）来创建一个经过优化的、tree-shaking 的生产包，以获得最佳的加载性能。

5. Webpack 的 Loader 和 Plugin 有什么不同？你自己写过吗？

Loader (加载器)：

- **作用**：Loader 是 **文件转换器**。
- **核心职责**：Webpack 默认只认识 JavaScript 和 JSON 文件。Loader 的作用是让 Webpack 能够去处理 *其他类型的文件*（如 `.css`、`.scss`、`.png`、`.vue`），并将它们转换为 Webpack 能理解的有效模块（通常是 JS 字符串或模块）。
- **运行机制**：它在 *模块加载时*（`import` 或 `require` 时）运行。它是一个函数，输入是源文件内容，输出是转换后的内容。
- **例子**：`babel-loader`（转换 ES6+ 到 ES5）、`css-loader`（处理 CSS 中的 `@import` 和 `url()`）、`style-loader`（将 CSS 注入到 DOM 的 `<style>` 标签中）、`vue-loader`（处理 `.vue` 单文件组件）。
- **配置**：在 `module.rules` 数组中配置。

Plugin (插件):

- **作用:** Plugin 是**功能扩展器**。
- **核心职责:** Plugin 可以贯穿 Webpack 打包的**整个生命周期**。它可以在 Webpack 运行的特定“钩子 (Hook)”上执行更广泛的任务，不限于转换单个文件。
- **运行机制:** 它通过监听 Webpack 广播的事件来工作，可以在编译 (compilation) 的各个阶段执行自定义逻辑。
- **例子:**
 - `HtmlWebpackPlugin`: 在打包结束后，自动生成一个 `index.html` 文件并引入打包好的 JS/CSS。
 - `MiniCssExtractPlugin`: 将 CSS 从 JS 中提取出来，生成单独的 `.css` 文件。
 - `TerserWebpackPlugin` 或 `UglifyJsPlugin`: 在打包**完成前**，对 JS 代码进行压缩和混淆。
- **配置:** 在顶层的 `plugins` 数组中配置 (通过 `new` 实例化)。

总结:

- **Loader** 负责“**转换**”，它在模块层面上工作 (`module.rules`)。
- **Plugin** 负责“**增强**”，它在构建的生命周期层面上工作 (`plugins`)。

(关于是否写过)

(建议回答): "我没有从零开始编写过复杂的 Loader 或 Plugin，因为社区生态已经非常成熟。但我深入研究过它们的工作原理，比如 style-loader 和 css-loader 的配合机制。我也根据项目需求修改过一些 Loader 的配置，并且能熟练配置各种常用的 Plugin 来优化构建流程。"

6. 从输入 URL 到页面完全加载，中间发生了什么？

这是一个非常经典的面试题，涵盖了网络和浏览器渲染的全部流程：

1. **URL 解析:** 浏览器首先解析用户输入的 URL，判断协议 (如 `HTTPS`)、域名、端口、路径等。
2. **DNS 查询:**
 - 浏览器会检查本地缓存 (浏览器缓存、系统缓存、路由器缓存) 中是否有这个域名对应的 IP 地址。
 - 如果缓存中没有，浏览器会向本地 DNS 服务器发起请求，通过递归或迭代查询，最终从权威 DNS 服务器获取到域名对应的 IP 地址。
3. **建立 TCP 连接:** 浏览器拿到 IP 地址后，向服务器发起 TCP 连接请求。这个过程会经历“三次握手”，以确保客户端和服务端都准备好通信。
4. **TLS/SSL 握手:** 如果是 `HTTPS` 协议，在 TCP 连接建立后，还需要进行 SSL/TLS 握手。这个过程用于交换加密密钥、验证服务器证书，以建立一条安全的加密通道。

5. **发送 HTTP/HTTPS 请求：**浏览器构建一个 HTTP 请求报文（包括请求行、请求头 `Headers`，如 `Cookie`、`User-Agent` 等），并通过 TCP 连接发送给服务器。
 6. **服务器处理请求并响应：**服务器（如 Nginx、Node.js）接收到请求，进行处理（可能涉及后端路由、数据库查询），然后构建一个 HTTP 响应报文（包括状态码如 `200 OK`、响应头 `Headers`，如 `Cache-Control`、`Content-Type`，以及响应体 Body，即 HTML 内容）。
 7. **浏览器接收响应：**浏览器接收到服务器的响应。
 8. **（关键步骤）浏览器渲染页面：**
 - **解析 HTML (DOM)：**浏览器自上而下解析 HTML 响应体，开始构建 **DOM 树 (Document Object Model)**。
 - **解析 CSS (CSSOM)：**当解析器遇到 `<link>` 或 `<style>` 标签时，会去请求并解析 CSS，构建 **CSSOM 树 (CSS Object Model)**。
 - **构建渲染树 (Render Tree)：**浏览器将 DOM 树和 CSSOM 树结合起来，生成渲染树。渲染树只包含需要显示的节点（比如 `display: none` 的节点不会在渲染树中）。
 - **布局 (Layout/Reflow)：**浏览器根据渲染树，计算出每个节点在屏幕上的确切位置和大小。
 - **绘制 (Paint)：**浏览器调用 GPU，将各个节点绘制到屏幕上。
 - **合成 (Compositing)：**对于复杂的页面（比如有 `transform` 或 `opacity`），浏览器会将页面分成多个图层，单独绘制，最后再合成到一起。
 9. **处理 JavaScript：**
 - 当 HTML 解析器遇到 `<script>` 标签时，它会**暂停 HTML 解析**。
 - 它会立即下载、解析并执行 JavaScript。
 - JS 可能会操作 DOM 和 CSSOM（比如 `document.getElementById`）。如果 JS 修改了 DOM 或 CSSOM，浏览器可能需要重新执行布局 (Reflow) 和绘制 (Repaint)。
 - (`defer` 和 `async` 属性可以改变 JS 的加载和执行时机，以优化性能)。
-

10. 了解浏览器的强缓存和协商缓存吗？

了解。这是浏览器缓存的两种主要策略，都是为了减少网络请求，加快资源加载。当浏览器请求一个资源时，它会按以下顺序工作：

11. 强缓存 (Strong Cache)：

- **机制：**浏览器不向服务器发送任何请求，而是直接从本地缓存中读取资源。
- **判断依据：**浏览器检查本地缓存资源的 `Headers`。
 - **`Cache-Control` (HTTP/1.1, 优先级高)：**这是一个相对时间。
 - `max-age=3600`：表示这个资源在 3600 秒（1小时）内是新鲜的，直接使用缓存。

- `no-cache`：字面意思是“不要缓存”，但实际上是“跳过强缓存”，必须进入协商缓存。
- `no-store`：最严格，禁止浏览器缓存这个资源。
- **Expires (HTTP/1.0)**：这是一个绝对的过期时间（GMT 格式）。如果 `Cache-Control` 和 `Expires` 同时存在，`Cache-Control` 优先。
- **状态码**：如果强缓存命中，浏览器从本地读取，网络（Network）面板会显示 `(from memory cache)` 或 `(from disk cache)`，状态码为 **200**。

12. 协商缓存（Negotiation Cache / ETag）：

- **机制**：当强缓存失效（比如 `max-age` 过期了），或者 `Cache-Control` 被设为 `no-cache` 时，浏览器必须向服务器发送一个请求，但这个请求会带上“缓存标识”。
- **服务器判断**：服务器根据浏览器带来的“缓存标识”，判断资源是否真的更新了。
 - **如果未更新**：服务器返回 **304 Not Modified** 状态码，响应体为空。浏览器收到 304 后，就知道本地缓存依然可用，于是从本地读取。
 - **如果已更新**：服务器返回 **200 OK** 状态码，并附带新的资源内容和新的缓存标识。
- **缓存标识（成对出现）**：
 - a. **Last-Modified / If-Modified-Since**：
 - 服务器在上次响应时，通过 `Last-Modified` 告诉浏览器资源的最后修改时间。
 - 浏览器在协商缓存请求时，通过 `If-Modified-Since` 把这个时间带回去。
 - 服务器比较这个时间和文件的真实修改时间。
 - （缺点：时间戳只能精确到秒；如果文件内容没变但时间戳变了，也会重新下载）。
 - b. **ETag / If-None-Match**（优先级高）：
 - `ETag` 是服务器为资源生成的唯一标识（类似文件指纹或 Hash）。
 - 服务器在上次响应时，通过 `ETag` 告诉浏览器这个标识。
 - 浏览器在协商缓存请求时，通过 `If-None-Match` 把这个 `ETag` 带回去。
 - 服务器比较这个 ETag 和文件当前的 ETag 是否一致。
 - （优点：`ETag` 更精确，能解决 `Last-Modified` 的问题）。

二面参考答案

1. 前端性能优化你一般从哪些方面入手？

我会从两个主要阶段来看待性能优化：**加载阶段**和**运行时阶段**。

加载阶段（网络优化）：目标是让资源更快、更小地到达浏览器。

- **减少请求体积：**
 - **压缩：** HTML、CSS、JS 文件进行 Gzip 或 Brotli 压缩。
 - **图片优化：** 使用 WebP 或 AVIF 格式，对 PNG/JPG 进行压缩。
 - **代码精简：** JS、CSS、HTML 的 Minify（最小化），以及 Tree-shaking 移除死代码。
- **减少请求数量：**
 - **打包：** 使用 Webpack/Vite 将多个 JS/CSS 文件打包。
 - **雪碧图 (Sprites)：** 合并小图标（虽然在 HTTP/2 下优先级降低了）。
 - **内联：** 将首屏关键的、体积很小的 CSS 或 JS 直接内联到 HTML 中。
- **加快请求速度：**
 - **CDN：** 将静态资源部署到 CDN，利用边缘节点就近访问。
 - **HTTP/2：** 开启 HTTP/2，利用多路复用特性。
 - **缓存：** 合理设置 `Cache-Control` 和 `ETag`，充分利用强缓存和协商缓存。
 - **DNS 预解析：** 使用 `rel="dns-prefetch"`。

运行时阶段（渲染和执行优化）： 目标是让页面交互更流畅。

- **渲染优化：**
 - **减少重排 (Reflow) 和重绘 (Repaint)：** 比如批量修改 DOM；使用 `transform` 和 `opacity` 触发 GPU 加速的合成 (Compositing)，避免触发布局。
 - **CSS 优化：** 避免复杂的 CSS 选择器。
- **JS 执行优化：**
 - **Code Splitting（代码分割）：** 按路由或按组件懒加载 JS，减少首屏 JS 的解析和执行时间。
 - **长列表优化：** 使用虚拟滚动或无限滚动。
 - **防抖 (Debounce) 和节流 (Throttle)：** 用于 `scroll`、`resize`、`input` 等高频事件。
- **资源加载时机：**
 - **JS 加载：** 使用 `defer` 或 `async` 属性，避免 JS 阻塞 DOM 解析。
 - **图片懒加载：** 对视口外的图片使用 `loading="lazy"` 属性。

2. LCP、FCP、CLS 这些指标分别指什么？你怎么去监控和优化？

这些是 Google 提出的核心 Web 指标 (Core Web Vitals)，用于衡量用户体验。

- **FCP (First Contentful Paint, 首次内容绘制)：**
 - **指什么：** 浏览器渲染出第一个 DOM 内容（文本、图片、Canvas 等）的时间点。它衡量的是“加载开始了”的感觉。

- **LCP (Largest Contentful Paint, 最大内容绘制):**

- **指什么:** 视口内最大的可见内容（通常是主图或大段文本）被渲染出来的时间点。它衡量的是“主要内容加载完了”的感觉。

- **CLS (Cumulative Layout Shift, 累积布局偏移):**

- **指什么:** 测量页面在加载过程中, 可见元素发生“意外”位置偏移的累积分数。它衡量的是“页面稳定性”。比如你正要点击一个按钮, 结果上面突然加载出一个广告, 导致按钮下移。

如何监控:

1. **本地/实验室数据:** 使用 Chrome DevTools 里的 **Lighthouse** 面板, 可以跑分并查看这些指标。

2. **线上/真实用户数据 (RUM):**

- 使用 Google 的 `web-vitals` 库, 它很轻量, 可以在用户端收集数据。
- 将收集到的数据上报到我们的监控平台 (比如 Sentry、或其他自研平台) 进行聚合分析。

如何优化:

- **优化 FCP/LCP (它们高度相关):**

- a. **优化 TTFB (Time to First Byte):** 减少服务器响应时间, 比如优化 API、使用 CDN。
- b. **消除渲染阻塞资源:** 非关键的 CSS 拆分出去异步加载, JS 使用 `defer` 或 `async`。
- c. **优化 LCP 元素:** 如果 LCP 元素是图片, 要确保它被优先加载 (比如使用 `preload`), 并且经过了充分压缩。

- **优化 CLS:**

- a. **给图片/视频设固定尺寸:** 在 CSS 中为 `<img>`、`<video>` 设置明确的 `width`、`height` 或使用 `aspect-ratio` 属性。
- b. **预留空间:** 对于动态插入的内容 (如广告、Banner), 提前在 DOM 中预留好固定大小的占位符。
- c. **避免在现有内容上方插入新内容:** 除非是响应用户操作。

3. 如果一个列表页数据量特别大, 一次性加载很慢, 你会怎么优化?

一次性加载的瓶颈在于两点: 一是 API 请求数据量大, 二是 DOM 元素过多导致渲染卡顿。我会根据场景选择不同的策略:

1. 分页 (Pagination):

- 这是最传统和最简单的方案。只向后端请求一页的数据 (比如 20 条), 在页面底部提供页码导航。
- **优点:** 简单可靠, 用户对总数有预期。

- **缺点：** 交互上需要用户额外点击，不适用于“无限信息流”场景。

2. 无限滚动 (Infinite Scrolling):

- 这是移动端 C 端业务很常见的方案。首次加载第一页数据，当用户滚动到列表底部时，再触发 API 请求下一页数据，并追加到列表末尾。
- **实现：** 我会使用 `IntersectionObserver` API 来监听列表底部的“加载更多”元素是否进入视口，以此来触发加载。
- **缺点：** 随着用户不断滚动，DOM 元素会无限累积，最终还是会导致页面卡顿和内存占用过高。

3. 虚拟滚动 (Virtual Scrolling):

- 这是解决海量 DOM 渲染的终极方案。

（如果面试官问“无限滚动和虚拟滚动的区别”）：无限滚动解决的是数据加载问题（分批加载数据），但没有解决 DOM 渲染问题；虚拟滚动解决的是 DOM 渲染问题（只渲染可见区域）。

4. 什么是虚拟滚动？它的实现原理是什么？

虚拟滚动（也叫“窗口化”）是一种只渲染用户当前可见区域内的列表项，以实现在有限的 DOM 元素下展示海量数据的技术。

实现原理：

它的核心思想是“只渲染视口内和视口上下一小部分缓冲区的内容”。

1. **容器 (Container):** 需要一个固定高度的、`overflow: auto` 的外层容器，用来产生滚动条。
2. **滚动占位 (Scroller/Phantom):** 在容器内部，我们需要一个“幽灵”元素。这个元素本身不可见，但它的**总高度**被设置为**所有列表项的总高度**（例如，10万条数据 * 50px/条 = 5,000,000px）。它的唯一作用就是撑开容器，产生一个“正确”的滚动条。
3. **内容窗口 (Window):** 在容器内部（与“幽灵”元素同级或嵌套），我们还需要一个“内容窗口”。这个窗口通常使用绝对定位（`position: absolute`），它只承载**当前可见**的那些 DOM 元素（比如只渲染 20 条）。
4. **滚动监听与计算：**
 - 监听外层容器的 `scroll` 事件。
 - 当滚动发生时，根据 `scrollTop`（滚动条的垂直偏移）和 每一项的固定高度，计算出当前应该在视口中的**数据索引**（`startIndex`）。
 - `startIndex = Math.floor(scrollTop / itemHeight)`。
5. **数据切片与定位：**
 - 根据 `startIndex` 和视口能容纳的条目数，从完整的数据源中 `slice` 出需要渲染的数据。
 - 更新“内容窗口”的 DOM 节点。

- 同时，使用 CSS `transform: translateY(Y_OFFSET)` 来偏移“内容窗口”，使其在视口中保持正确的位置。这个 `Y_OFFSET` 通常就是 `startIndex * itemHeight`。

这样，无论用户怎么滚动，DOM 元素的数量始终是恒定的，只是在动态更新数据和移动位置。

6. 看 JD 了吧？我们 C 端业务对 SEO 有要求。你之前做过 SEO 优化吗？具体做了哪些？

是的，做 C 端业务，SEO 是非常重要的的一环。我之前在 XXX 项目中，主要从以下几个方面做了优化：

1. 渲染方案（最重要）：

- 我们知道，标准的 Vue/React 客户端渲染（CSR）对爬虫非常不友好，爬虫抓到的是一个空壳 HTML。
- 为了解决这个问题，我们采用了 **SSR（服务端渲染）**（或者 SSG 静态站点生成）。
- （如果用了 SSR 框架）我们使用了 Nuxt.js / Next.js，它能确保服务器在响应请求时，返回的就是包含完整页面内容的 HTML，这样爬虫就能抓取到所有内容。

2. TDK 优化：

- 为每个页面动态设置 `Title`（标题）、`Description`（描述）、`Keywords`（关键词）。
- 在 SSR 框架里，我们会在页面的 `head` 方法（或 `useHead`）中，根据当前页面的数据动态生成 TDK。

3. 语义化 HTML：

- 严格使用 `h1`、`h2`、`h3` 等标签来组织内容层级；使用 `nav`、`main`、`article` 等语义化标签。
- 确保图片 `<img>` 标签都有 `alt` 属性，为爬虫提供图片信息。

4. Crawlability（可爬取性）：

- sitemap.xml**：我们会用脚本自动生成 `sitemap.xml` 并提交给搜索引擎，告诉它们网站有哪些页面。
- robots.txt**：用来告诉爬虫哪些路径不要抓取（比如用户后台、购物车）。
- 内链**：确保网站的页面间有合理的 `<a>` 标签链接，而不是用 JS `onClick` 跳转，这样爬虫才能顺着链接遍历整站。

5. 结构化数据 (Structured Data)：

- （如果做过）对于商品或文章页面，我们会使用 JSON-LD 格式，在 HTML 中注入结构化数据，帮助搜索引擎理解页面内容，使其在搜索结果中展示“富摘要”（Rich Snippets）。

6. SSR 和 CSR 的区别是什么？

它们是两种完全不同的页面渲染模式。

- **CSR (Client-Side Rendering, 客户端渲染):**

- **流程:**

- i. 浏览器请求 URL。
 - ii. 服务器返回一个几乎空的 **HTML** (比如只有一个 `<div id="app"></div>`) 和一个完整的 **JS 包**。
 - iii. 浏览器下载并执行这个 JS 包。
 - iv. JS (Vue/React) 开始在客户端 (浏览器) 执行, 拉取 API 数据, 然后渲染出页面内容。

- **优点:** 服务器压力小; 前后端分离彻底; 页面间切换快 (因为只是 JS 在切换, 不是整页刷新)。

- **缺点:** 首屏加载慢 (FCP/LCP 高), 用户会看到白屏; **SEO 极差**, 爬虫抓不到内容。

- **SSR (Server-Side Rendering, 服务端渲染):**

- **流程:**

- i. 浏览器请求 URL。
 - ii. 服务器在 *Node.js* 环境中执行 Vue/React 代码, 拉取 API 数据, 在服务端就渲染出完整的 **HTML 字符串**。
 - iii. 服务器将这个包含内容的 **HTML** 返回给浏览器。
 - iv. 浏览器立即可以显示页面内容 (FCP/LCP 快)。
 - v. 同时, 浏览器下载 JS 包。
 - vi. JS 执行, 接管 ("Hydrate", 注水) 页面, 使其变得可交互。

- **优点:** 首屏加载快; **SEO 友好**。

- **缺点:** 服务器压力大; 架构更复杂; 开发时要考虑代码在 Node.js 和浏览器两端执行的兼容性。

7. 你们团队的通用组件是怎么沉淀的? 如何保证组件的质量?

组件沉淀的流程:

我们不是为了抽组件而抽组件, 组件沉淀一般遵循“自下而上”的流程:

1. **发现:** 在业务开发中, 发现某个模块 (比如一个带搜索和分页的表格、一个城市选择器) 在 A、B 两个项目中都用到了。
2. **抽象:** 确定它是一个“通用需求”。我们会指定一个同学, 将这个组件从具体业务中剥离出来, 把业务逻辑 (如特定的 API 请求) 通过 `props`、`events` 或 `slots` 的方式暴露出去。
3. **评审:** 抽象完成后, 我们会进行一次小型的代码评审 (Code Review), 确保这个组件的 API 设计是合理的、可扩展的。

4. **入库**：评审通过后，我们将它放入团队的**私有 NPM 仓库**（或者 Monorepo 里的 `shared` 包），并更新版本号。

保证组件质量的手段：

1. **TypeScript（类型安全）**：所有的通用组件必须使用 TypeScript 编写，`props` 和 `events` 必须有严格的类型定义。这是最基础的质量保证。
2. **文档与示例（可维护性）**：
 - 我们会用 Storybook 或类似的工具为组件编写文档。
 - 文档必须包含：清晰的 API 说明（每个 `prop` 的作用、类型、默认值）、至少一个可交互的 Demo。
3. **单元测试（稳定性）**：
 - 复杂组件（比如有内部状态和复杂逻辑的）必须编写单元测试（使用 Vitest 或 Jest）。
 - 测试会覆盖组件的核心逻辑和边界情况。
4. **代码规范（一致性）**：
 - 遵循团队统一的 ESLint 和 Prettier 规范。
5. **版本控制（可追溯性）**：
 - 严格遵循 SemVer (语义化版本) 规范。修复 Bug 发 patch 版，新增功能发 minor 版，有 Breaking Change 必须发 major 版。

6. 看你用过 TypeScript, interface 和 type 有什么区别？

它们在很多情况下可以互换，比如定义一个对象：

TypeScript

代码块

```
1 interface User {  
2   id: number;  
3 }  
4 type User = {  
5   id: number;  
6 };
```

这两种写法效果几乎一样。但它们有几个关键区别：

1. **可扩展性（Declaration Merging）**：
 - `interface` 支持**声明合并**。如果你在同一个作用域定义了两个同名的 `interface`，它们会自动合并。

- `type` **不支持**声明合并，如果定义两次同名的 `type`，TS 会直接报错。
- **场景：**这个特性使得 `interface` 更适合“扩展”第三方的类型。比如我想给 `window` 对象加个属性，我就可以用 `interface` 合并：
- TypeScript

代码块

```
1 interface Window { myProp: string; }
2 // 现在 window.myProp 就有类型提示了
```

2. 可定义类型的范围：

- `type` 更灵活。`type` 可以定义**任何**类型，包括联合类型、交叉类型、元组、基本类型别名。
- TypeScript

代码块

```
1 type Status = "success" | "error"; // 联合类型
   type UserID = string; // 基本类型别名
   type Point = [number, number]; // 元组
```

- `interface` 只能用来声明**对象（Object）或类（Class）**的结构。

3. 继承/实现：

- `interface` 使用 `extends` 关键字来继承。
- `type` 使用 `&`（交叉类型）来合并。
- `class` 可以 `implements` 一个 `interface`，也可以 `implements` 一个 `type`（但这个 `type` 必须是对象结构）。

总结我的使用习惯：

- 当定义**公共 API 的数据结构、对象或类**时，我**优先使用** `interface`，因为它支持声明合并，扩展性更好。
- 当定义**联合类型、元组或函数类型**时，我**必须使用** `type`。

4. 移动端 H5 开发和 PC 开发有什么主要差异？怎么做适配？

主要差异：

1. **交互方式：**PC 是鼠标交互，有 `hover`、`click`、`mousemove`。H5 是触摸交互，主要是 `touchstart`、`touchend`、`touchmove`，**没有** `hover`。这导致很多 PC 上的交互（如下拉菜单）在 H5 上要重新设计。

2. **运行环境：** PC 几乎都是标准浏览器。H5 除了标准浏览器，更多是运行在 App 的 **WebView** 中（如微信、企业 App），这会涉及 **JSBridge** 和原生 App 的通信。
3. **屏幕/视口：** PC 屏幕大，视口基本固定。H5 屏幕小，尺寸碎片化严重，并且有高 DPI（Retina）屏幕，会导致 1px 边框问题。
4. **性能/网络：** H5 设备的性能和网络环境（3G/4G/5G）通常比 PC 差，对性能优化（包体积、首屏加载）的要求更苛刻。

H5 适配方案：

适配的核心是让我们在设计稿（比如 750px 或 375px）上写的 `px` 单位，能自动缩放到不同尺寸的屏幕上。

1. Viewport Meta 标签（基础）：

- 首先必须设置这个标签：
- `<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">`

2. 主流适配方案（二选一）：

◦ 方案一：REM 适配（最常用）

- **原理：** `rem` 是相对于 `<html>` 元素的 `font-size` 来计算的。
- **做法：** 我们用一个 JS 脚本（比如 `amfe-flexible`）在页面加载时，根据当前设备的宽度（`document.documentElement.clientWidth`）动态计算并设置 `<html>` 的 `font-size`。
- 比如 750px 宽的设计稿，我们规定 `<html>` `font-size` 为 100px，那么设计稿上 200px 的元素，CSS 就写 `2rem`。当屏幕宽度变为 375px 时，JS 自动把 `<html>` `font-size` 设为 50px，`2rem` 就会自动缩放为 100px。
- 为了方便，我们通常会用 `postcss-pxtorem` 插件，它在打包时自动把我们写的 `px` 转成 `rem`。

◦ 方案二：VW 适配（更现代）

- **原理：** `vw` 是视口宽度的单位，`100vw` = 视口宽度。
- **做法：** 750px 宽的设计稿，`100vw` = 750px。我们直接使用 `postcss-px-to-viewport` 插件，在打包时自动把 `px` 转成 `vw`。
- **优点：** 不需要 JS 脚本，纯 CSS 解决。
- **缺点：** 某些老机型（iOS 8-9）有兼容性问题，但现在基本可以忽略。

3. H5 页面白屏问题，你一般怎么排查？

白屏是 H5 开发最严重的问题，我的排查思路一般是：

1. **复现问题：**搞清楚是**所有用户**白屏，还是**特定机型、特定网络、特定 App 版本**才白屏。

2. 检查资源加载（Network）：

- 如果能复现，打开 DevTools 看 Network。
- 最常见的是**资源 404**。是不是打包时 `publicPath` 配置错了？或者 CDN 挂了？导致 `app.js` 或 `app.css` 加载失败。
- 如果是 5xx 错误，那是服务端挂了。

3. 检查 JS 运行时错误（Console）：

- 这是最可能的原因。**
- 检查 Console 面板，看是不是在 JS 初始化阶段（比如 `main.js` 或 `App.vue` 的 `created` 里）就抛出了一个**阻塞性错误**。
- 比如，代码里用了 `localStorage.getItem('user').name`，但 `user` 是 `null`，导致 `cannot read property 'name' of null`，整个 Vue/React 应用就崩溃了，无法挂载 DOM。

4. 检查 DOM 结构（Elements）：

- 看 Elements 面板，`<body>` 里面是不是只有一个空的 `<div id="app"></div>`？
- 如果是，说明 JS 确实执行失败了，没有把组件挂载上去。

5. 检查环境（特定 WebView）：

- 如果只在 App 内白屏，但在浏览器里正常。
- 我会怀疑是不是 JS **依赖了 App 提供的 JSBridge**，但 App 没有及时注入，或者注入失败。比如代码在 `onReady` 之前就调用了原生的方法，导致 JS 报错。

6. 查看线上监控：

- 如果本地无法复现，我会马上去看 **Sentry** 或 **Fundebug** 这样的线上错误监控平台。
- 根据用户反馈的时间点，过滤一下错误日志，通常能很快定位到是哪个 JS 错误导致了白屏。

7. 前端工程化方面，除了打包，你还做过哪些事情？比如 CI/CD、代码规范。

是的，打包只是工程化的一小部分。我之前的工作中，工程化实践覆盖了整个开发生命周期：

1. 代码规范（开发阶段）：

- ESLint + Prettier + Stylelint**：我们在项目中强制使用这套“全家桶”，`ESLint` 负责代码质量（比如“禁止使用 `var`”），`Prettier` 负责代码风格（比如“统一使用单引号和 2 个空格缩进”）。
- husky + lint-staged**：为了防止不规范的代码被提交，我们使用 `husky` 设置了 `Git pre-commit` 钩子。这个钩子会触发 `lint-staged`，它只对本次提交（`staged` 状态）的文件执行 `eslint --fix` 和 `prettier --write`。

- `commitlint`：我们还用了 `commitlint` 来规范 Git 提交信息，比如必须以 `feat:`、`fix:`、`docs:` 开头，这对后续生成 CHANGELOG 和版本回溯很有帮助。

2. CI/CD（集成与部署阶段）：

- 我们使用 **GitLab CI**（或 GitHub Actions）。
- **CI (Continuous Integration)**：当开发者提交 Merge Request 时，CI 流水线会自动触发。它会执行：
 - `npm install`（安装依赖）
 - `npm run lint`（检查代码规范）
 - `npm run test`（跑单元测试）
 - `npm run build`（确保项目能成功打包）
 - 只有当 CI 全部通过时，代码才允许被合并，这能有效防止“构建失败”的代码合入主分支。
- **CD (Continuous Deployment)**：当 MR 被合并到 `main` 或 `production` 分支时，CD 流水线会自动触发。它会执行：
 - `npm run build`（打包生产环境）
 - `docker build ...`（构建 Docker 镜像，如果是 Node.js 项目）
 - `upload-to-s3/oss`（将静态资源推送到 CDN）
 - `purge-cdn-cache`（刷新 CDN 缓存）
 - 这样就实现了自动化的、无人值守的部署。